

RQ2DSL: An AI Agent for Automated Domain-Specific Language Grammar Generation from Requirements

1st Theodoros Tsampouris

*Dept. of Electrical and Computer Engineering
Aristotle University of Thessaloniki
Thessaloniki, 54124, Greece
ttsampob@ece.auth.gr*

2nd Emmanouil Tsardoulis

*Dept. of Electrical and Computer Engineering
Aristotle University of Thessaloniki
Thessaloniki, 54124, Greece
etsardou@ece.auth.gr*

3rd Konstantinos Panayiotou

*Dept. of Electrical and Computer Engineering
Aristotle University of Thessaloniki
Thessaloniki, 54124, Greece
klpanagi@ece.auth.gr*

4th Andreas L. Symeonidis

*Dept. of Electrical and Computer Engineering
Aristotle University of Thessaloniki
Thessaloniki, 54124, Greece
symeonid@ece.auth.gr*

Abstract—Domain-Specific Language (DSL) design is a foundational yet labor-intensive task in software engineering (SE). However, recent advances in Large Language Models (LLMs) offer the potential to automate key aspects of DSL development, including grammar generation, paving the way for wider DSL adoption. This paper presents RQ2DSL, an AI agent for automated grammar generation that transforms requirements (Functional and Non-Functional) into syntactically correct DSL grammars using LLMs. The agent's methodology comprises an optimally defined LLM context that contains a TextX knowledge base, reference grammars, syntactic examples and error-aware guidance with explicit validation rules, supported by an iterative refinement process that leverages syntax validation feedback. Comprehensive experiments were conducted against DSLs of varying complexity levels (simple, medium, high, ultra-high), with scalable requirement sets ranging from 4 to 30 specifications, demonstrating that RQ2DSL maintains robust performance across simple (100%), medium (92%) and high (92%) complexity levels. However, a critical complexity threshold emerges at ultra-high complex DSLs, where RQ2DSL experiences a notable performance degradation to 75%, while reduced-context approaches lead to a dramatic drop in success rate.

Index Terms—Domain-specific languages, large language models, prompt engineering, grammar generation, model-driven engineering

I. INTRODUCTION

Domain-Specific Languages (DSLs) have become indispensable tools in modern software engineering, enabling domain experts to express solutions using familiar terminology while maintaining formal computational rigor. By abstracting away low-level implementation details, DSLs bridge the gap between domain knowledge and software implementation, facilitating more intuitive and maintainable code development across diverse application areas [1]. However, the creation of syntactically correct and semantically meaningful DSL grammars remains a complex, time-intensive process that typically

requires specialized expertise in both the target domain and language engineering principles [2].

Traditional DSL development follows a laborious iterative process involving domain analysis, grammar specification, parser generation and extensive validation. This workflow often creates bottlenecks in DSL development, as it requires close collaboration between domain experts who understand the problem space and technical experts who possess the necessary language engineering skills [3]. The complexity of this process has historically limited the widespread adoption of DSLs, despite their proven benefits in improving code clarity, reducing complexity development time and minimizing domain-specific errors [4].

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in code generation and understanding, fundamentally transforming approaches to automated software development [3], [5], [6]. Models like Codex [5] have shown strong capabilities in generating functionally correct programs across multiple programming languages, while sophisticated prompt engineering techniques such as grammar prompting and chain-of-thought reasoning have enhanced LLM performance on structured generation tasks [7], [8]. These developments suggest that LLMs could potentially automate significant portions of the DSL generation process.

However, despite these advancements in generative AI for code generation, there remains a significant gap in practical tools for complete DSL grammar generation across varying complexity levels, leaving practitioners without automated solutions for DSL development workflows. While existing research has focused primarily on semantic parsing within established DSLs or generating simple code fragments, comprehensive grammar generation tools remain largely unavailable.

The TextX meta-language provides an ideal foundation for

this investigation, offering a Python-based meta-language with a grammar-first approach and robust validation mechanisms for assessing syntactic correctness [9]. This paper presents RQ2DSL, an AI agent that implements a systematic methodology for automated PEG grammar generation, transforming requirements into syntactically correct DSL grammars using LLMs. RQ2DSL incorporates an optimal context specification encapsulating TextX syntax specifications, reference grammars, syntactic examples and error-aware guidance, supported by iterative refinement through syntax validation feedback. Our contributions include: (1) RQ2DSL, a practical tool for requirements-to-grammar (R2G) transformation with clear complexity boundaries for practitioners and, (2) experimental validation demonstrating the tool’s effectiveness and limitations across varying DSL complexity levels. This work establishes boundaries for current LLM capabilities while providing immediate practical value through a deployable AI agent for DSL development workflows.

II. RELATED WORK

The intersection of LLMs and domain-specific language development builds upon three foundational research areas: (a) LLM capabilities for code and grammar generation, (b) prompt engineering techniques for structured tasks, and (c) automated DSL development methodologies and tools. This section examines the state-of-the-art in each area and positions RQ2DSL within the existing tool landscape.

A. LLMs for Code and Grammar Generation

Recent breakthroughs in neural code generation have demonstrated that large language models can produce syntactically correct code across multiple programming languages, establishing the foundation for more specialized applications such as DSL grammar generation. Chen et al. [5] introduced Codex, achieving a 28.8% success rate on the HumanEval benchmark and establishing fundamental evaluation methodologies that underpin practical tools like GitHub Copilot.

Building upon these foundations, researchers have developed sophisticated approaches for constraining LLM outputs to conform to formal specifications. Wang et al. [7] proposed grammar prompting for domain-specific language generation, using Backus-Naur Form (BNF) grammars to guide LLMs through a two-step process of predicting specialized grammars and generating conforming outputs. Their approach demonstrated competitive performance on semantic parsing tasks without requiring fine-tuning, providing a direct conceptual template for TextX grammar generation.

More recently, SynCode [10] ensures syntactic correctness through grammar augmentation, constructing offline Deterministic Finite Automaton (DFA) mask stores for efficient token filtering during generation. These DFAs, derived from Context-Free Grammar (CFG) specifications that formally define programming language syntax rules, enable the framework to achieve a 96.07% reduction in syntax errors for Python and Go code generation while remaining model-agnostic and applicable to any CFG-defined language. PICARD [11] on

the other hand, proposes incremental parsing for constrained auto-regressive decoding, rejecting inadmissible tokens at each step and achieving state-of-the-art performance on text-to-SQL tasks, demonstrating how grammar-constrained decoding generalizes to structured language domains.

In parallel, syntax-aware neural models have been applied to structured code generation tasks. Brad et al. [12] proposed a syntax-guided neural model for generating SQL queries from natural language by constructing Abstract Syntax Trees (ASTs) based on the underlying SQL grammar. Their approach ensures syntactic correctness by decoding structured ASTs rather than flat token sequences. While their work focuses on mapping natural language queries to existing DSLs like SQL, it shares methodological similarities with RQ2DSL in using grammar-constrained decoding to ensure syntactic validity. However, our approach targets the generation of new DSL grammars from requirements, rather than generating valid programs within predefined languages.

B. Prompt Engineering Techniques for Structured Tasks

The development of sophisticated prompting strategies has proven crucial for eliciting structured outputs from LLMs. Chain-of-thought prompting with few-shot demonstrations significantly improves model performance on complex tasks, especially for large models exceeding 50 billion parameters [13]. The Structured Chain-of-Thought (SCoT) approach [8] organizes reasoning by program constructs, demonstrating up to 13.79% improvement in Pass@1 (the percentage of problems solved correctly on the first attempt) over standard prompting on established benchmarks. These insights prove directly applicable to DSL grammar generation, where structural reasoning about language constructs is essential for producing syntactically valid outputs.

Iterative refinement has emerged as another powerful strategy for improving generation quality. Self-Refine [14] incorporates self-feedback through a refinement loop, producing gains of 5–40% over direct generation without requiring supervised training data. This strategy proves particularly relevant to DSL grammar generation, where feedback from parser errors can inform iterative grammar corrections, aligning closely with our error-aware prompting methodology.

C. Automated DSL Development

Traditional approaches to automated DSL development heavily relied on language workbenches and meta-modeling frameworks, which can now be extended through LLM integration. The TextX meta-language [9] provides the core infrastructure for our work, offering a grammar-first approach for building DSLs through declarative specifications, supported by automatic parser and meta-model generation. Built on top of the Arpeggio PEG parser, TextX supports modular grammar definitions and extensible language composition, making it an ideal testbed for LLM-assisted grammar generation.

Building on these foundations, researchers have explored various approaches to automate different aspects of DSL

generation. Work by Ojamaa et al. [15] addressed the alignment of domain ontologies with DSL meta-models through a semi-automated transformation pipeline from OWL ontologies. This approach facilitates early-stage DSL design using formal knowledge representations, demonstrating an early effort toward automated DSL generation that our LLM-driven methodology extends with statistical generation capabilities.

The development of compositional language workbenches has further advanced automation possibilities. MontiCore [16] introduced support for reusable language components, inheritance and embedding, with grammar formats that enable automatic generation of parsers, abstract syntax trees, symbol tables and pretty printers. This modular approach illustrates benefits that LLMs could potentially enhance by automating component selection and completion processes.

Similarly, the Melange meta-language [17] supports modular DSL generation through type-safe reuse of existing components, providing a model typing system and algebraic operators for composing, extending, or restricting language features. These modularity mechanisms align well with using LLMs to suggest composable grammar fragments or automate DSL customization based on evolving domain requirements, creating a natural bridge between traditional workbench approaches and modern AI-assisted development.

In [18], Jordi Cabot et al. demonstrated the practical feasibility of using Claude 4 to create complete DSLs, including both ANTLR and TextX grammar generation for a funding specification language. While this work validates the potential of LLMs for DSL generation, it highlights the need for systematic prompting methodologies to optimize generation quality and reliability across varying complexity levels.

The convergence of these research areas suggests that while traditional language workbenches provide robust and modular infrastructures, LLM-based approaches can automate substantial portions of the grammar generation process through systematic methodologies implemented in practical tools. However, comprehensive tools for transforming requirements directly into DSL grammars (and not models of existing grammars) across varying complexity levels remain absent from the literature. This work addresses this gap by presenting RQ2DSL, an AI agent that implements a systematic methodology for automated R2G transformation using LLMs, providing practitioners with a tool that converts formal specifications into syntactically correct TextX grammars while establishing the boundaries of current language model capabilities in automated DSL development.

III. RQ2DSL INTERFACE

RQ2DSL provides practitioners with an intuitive web-based interface that streamlines the grammar generation process through three core screens designed for optimal user experience and workflow efficiency. The requirements specification interface (Fig. 1) enables users to define their DSL name and specify functional and non-functional requirements using natural language descriptions. The interface supports dynamic requirement addition and deletion, with each requirement

following the pattern "The grammar should support..." for consistency in LLM processing.

Fig. 1. RQ2DSL Requirements Specification Interface showing DSL name input and dynamic requirements definition.

Upon requirement submission, RQ2DSL presents the generated TextX grammar through an integrated code editor (Fig. 2) with real-time syntax validation and direct editing capabilities. Users can download the generated grammar as a .tx file for immediate integration into their development workflows.

Fig. 2. Grammar Editor Interface displaying the generated TextX grammar with syntax highlighting and validation status.

Additionally, RQ2DSL includes a comprehensive library interface that provides grammar management with metadata display and standard CRUD operations (view, edit, duplicate, rename, delete), facilitating long-term project management and grammar reuse across different development contexts. The interface design makes RQ2DSL accessible to both domain experts and technical practitioners without requiring specialized knowledge of grammar engineering or LLM prompt construction.

IV. METHODOLOGY

This section presents the systematic methodology underlying the RQ2DSL agent for automated PEG grammar gen-

eration using TextX. RQ2DSL transforms requirements into syntactically correct DSL grammars through backend LLMs via a generation-validation-refinement workflow that leverages optimal contextual information to guide the LLM through the grammar generation process. Additionally, the methodology incorporates robust validation and error correction mechanisms in order to maximize DSL generation success.

A. Context Framework

The methodology's effectiveness relies on an integrated context framework that provides the LLM with essential information for accurate grammar generation. This framework comprises five core data/information components:

TextX Grammar Syntax Specifications: Complete syntax documentation for the TextX meta-language, including rule definitions, operators and structural patterns that ensure the LLM understands the target grammar formalism.

Functional and Non-Functional Requirements: Precisely specified DSL requirements that define the target language's intended functionality, with explicit syntactic constraints where necessary to guide implementation decisions.

Reference Grammars: Validated example TextX grammars that serve as syntactic and structural templates. These reference grammars represent real-world, complex DSL implementations [19], [20], providing concrete examples of TextX grammar design and syntax patterns along with rule structures that can guide LLM generation decisions.

Model Examples: Concrete model instances that demonstrate expected DSL usage patterns and proper instantiation of grammar rules. These examples help disambiguate requirement interpretations and demonstrate proper grammar application in realistic scenarios.

Error Prevention Guidance: Instructions to avoid common TextX syntax errors along with proper syntax specifications. This guidance addresses delimiter declaration errors and undefined rule references, which represent the most frequent syntax violations encountered in automated grammar generation.

B. Generation-Validation-Refinement Workflow

The proposed methodology follows the systematic iterative process illustrated in Fig. 3:

Initial Generation: The LLM receives a single generation prompt containing all context components and generates an initial TextX grammar based on the provided requirements.

Syntax Validation: The generated grammar undergoes validation using TextX developer tools, providing authoritative syntax verification.

Iterative Refinement: If validation fails, the system captures specific syntax error messages and resubmits the complete original prompt along with the error information and malformed grammar, with explicit instructions to fix the identified errors.

Termination Criteria: The process terminates under two conditions: (a) when three consecutive refinement attempts produce identical syntax errors, indicating that the LLM entered a repetitive error state, or (b) after ten total refinement attempts without achieving syntactic correctness.

C. Implementation Framework

We utilize Claude 3.7 as the target LLM, selected based on established performance in code generation tasks and demonstrated capabilities in structured text generation [21]. The prompt structure maintains consistent formatting to ensure reproducible results, with all context components integrated into a single prompt for each generation attempt. The validation process focuses exclusively on syntactic correctness to ensure generated grammars conform to TextX meta-language (thus verifying generated PEG grammars) without introducing subjective evaluation criteria. This approach provides clear, objective metrics for assessing methodology effectiveness.

V. EXPERIMENTS

To validate the effectiveness of our proposed methodology and demonstrate that our context framework can achieve automated PEG grammar generation, we conducted comprehensive experiments comparing our complete context against reduced-context variants across multiple complexity levels. These experiments were specifically designed to prove that each data component of our methodology's context framework contributes meaningfully to generation success rates.

A. Experimental Objectives

The experimental validation process addresses three research questions:

RQ1: How do different context configurations affect LLM success rates in TextX grammar generation?

RQ2: What is the relationship between DSL complexity and generation success across different context approaches?

RQ3: Is there a complexity threshold for current LLM capabilities, where they reach their limitations?

B. Domain Selection

All experimental DSLs focus exclusively on the algorithmic domain to create domain-specific languages for expressing computational procedures and programming constructs. This choice is made in order to ensure consistency and control for domain-specific knowledge variations that could confound experimental results. The algorithmic domain was selected for several key reasons: (1) it provides well-defined, unambiguous requirements that minimize interpretation variability, (2) it offers natural complexity progression from basic arithmetic operations to advanced programming constructs, (3) it eliminates domain-specific jargon that might bias LLM performance based on training data exposure, and (4) it enables objective evaluation of syntactic correctness without requiring deep domain expertise for validation.

C. Complexity-based evaluation

To systematically evaluate our methodology across different DSL sophistication levels, we employed a four-tier complexity evaluation schema based on the number of requirements the end user defines as the input of the procedure:

Low Complexity (4 requirements): Basic arithmetic and variable operations (calculatorDSL)

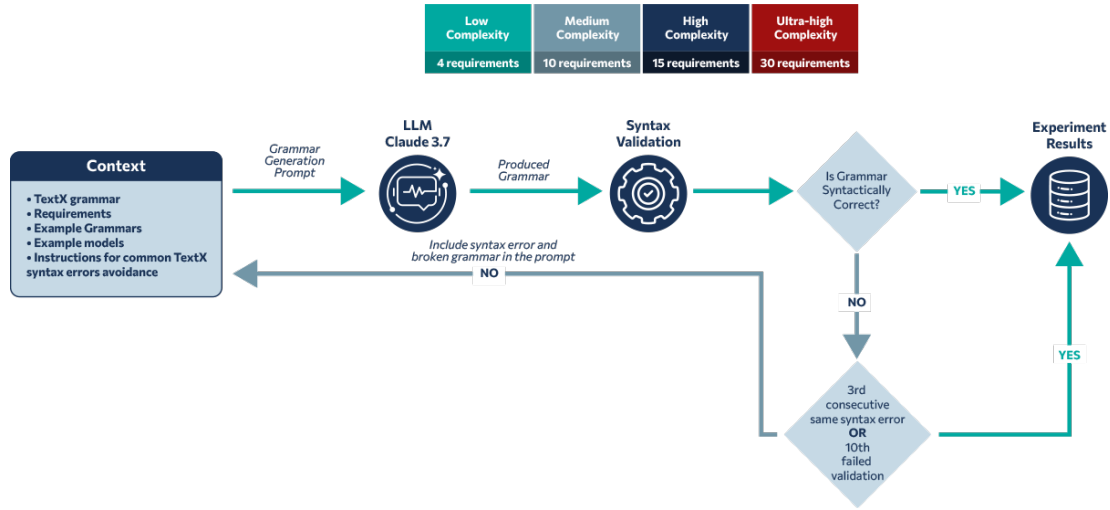


Fig. 3. Grammar Generation and Validation Flow

Medium Complexity (10 requirements): Configuration management with control flow (configurationDSL)

High Complexity (15 requirements): Algorithm specification with advanced control structures (pseudoDSL)

Ultra-high Complexity (30 requirements): Full-featured programming language constructs (enterpriseDSL)

The requirements for each complexity level are publicly available in our repository along with actual generated grammars for each one of the DSLs.¹

Each complexity level employs a cumulative approach, where requirements from previous levels are retained and augmented with additional functionality to progressively increase DSL sophistication. This approach enables systematic assessment of how methodology performance varies with increasing DSL complexity while maintaining consistent functional foundations across all test cases.

D. Context Variation Framework

To prove that our complete methodology actually needs all described context information, we systematically evaluated four different provided contexts by progressively removing context components:

Minimal Context (Baseline): Provides only fundamental context including TextX grammar generation task specification, official TextX grammar syntax, target DSL name and functional and non-functional requirements. This configuration establishes performance baselines without additional contextual guidance.

Reference-Enhanced Context: Adds validated example grammars to the minimal context, incorporating two professionally developed TextX grammars (goal-dsl.tx and sm-auto.tx) serving as syntactic and structural references.

Example-Rich Context: Extends the reference-enhanced approach with concrete model instances that demonstrate

expected DSL usage patterns and proper instantiation of grammar rules. These examples illustrate the intended syntax and usage without requiring validation against existing DSLs, serving as illustrative patterns for the target language design.

Complete Context (Our Methodology): Incorporates all previous components plus error prevention guidance for common TextX syntax errors, representing our complete proposed methodology.

This progressive context enrichment enables isolation of each component's contribution to generation success, demonstrating the value added by each element of our methodology's context framework.

E. Experimental Design and Execution

1) *Experimental Scale:* The validation employed differentiated sampling strategies based on complexity levels to balance statistical significance with computational feasibility:

Context Configurations 1-4: 100 experiments per DSL complexity level (Low, Medium, High)

Ultra-high Complexity: 20 experiments per context configuration

2) *Execution Protocol:* Each experiment follows the generation-validation-refinement workflow described in our methodology, with identical termination criteria and validation processes across all context configurations. The experimental protocol maintains a consistent LLM configuration (Claude 3.7), prompt formatting and validation procedures to ensure fair comparison between context variants.

All experiments utilize the same functional/non-functional requirements for each complexity level, with only the context framework components varying between different experimental conditions. This controlled approach ensures that observed performance differences can be attributed specifically to context variations rather than requirement complexity or implementation details.

¹<https://anonymous.4open.science/r/RQ2DSL-81F3>

3) *Validation and Quality Assurance*: The validation process employs the official TextX toolchain for authoritative syntax validation, focusing exclusively on syntactic validity to maintain objective evaluation criteria. Each generated grammar undergoes identical validation procedures regardless of the context configuration used for generation, ensuring consistent and unbiased assessment of methodology effectiveness across all experimental conditions.

VI. RESULTS

This section presents the experimental findings from 1,280 grammar generation experiments across four context configurations and four complexity levels.

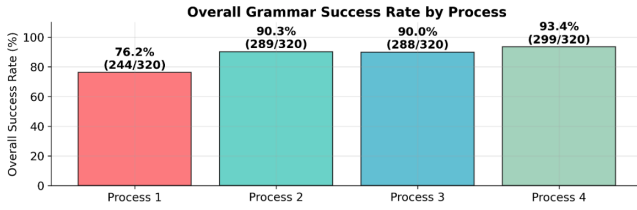


Fig. 4. Overall grammar syntax validation success rates across the four context configurations, demonstrating progressive improvement with diminishing returns from baseline to error-aware approaches.

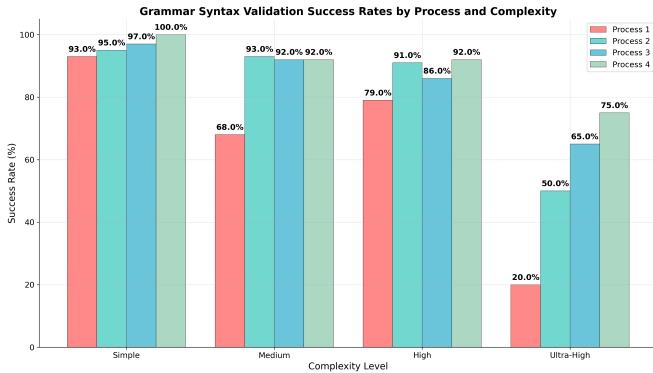


Fig. 5. Grammar syntax validation success rates across four context configurations and complexity levels, showing progressive improvement with enhanced context and dramatic performance degradation at ultra-high complexity.

A. Methodology Validation and Context Structure Evaluation

The experimental results demonstrate that our methodology (Context Configuration 4) provides optimal or equivalent performance for automated grammar generation across all complexity levels. As shown in Fig. 4, our methodology achieves the highest overall success rate at 93.4% when aggregated across all DSL complexity levels, representing a substantial 17.2 percentage point improvement over the baseline minimal context configuration (76.2%). The complexity-specific analysis in Fig. 5 reveals that our methodology achieves optimal performance at Low complexity (100%), matches the best alternatives at medium complexity (92%), provides optimal

performance at high complexity (92%) and demonstrates superior resilience at ultra-high complexity (75% compared to 20-65% for other approaches).

The progressive context enhancement reveals a clear pattern of improvement with diminishing returns. The reference-enhanced context (Configuration 2) achieves the largest single improvement, jumping from 76.2% to 90.3% overall success rate compared to the baseline. This 14.1 percentage point improvement represents the critical breakthrough in LLM grammar generation capabilities, demonstrating that providing concrete grammar examples significantly enhances generation quality. The subsequent configurations show more modest but meaningful gains, with the example-rich context (Configuration 3) achieving 90.0% and our complete methodology reaching 93.4% success rates.

These findings directly address **RQ1**, confirming that our methodology's context framework provides optimal value for automated grammar generation. While progressive context enhancement improves performance, the relationship exhibits diminishing returns rather than linear improvement, with reference grammar examples proving to be the most impactful single contextual enhancement.

B. Complexity-Dependent Performance Validation

The relationship between DSL complexity and generation success reveals that our methodology maintains superior performance across all complexity levels while demonstrating remarkable resilience at extreme complexity boundaries. As illustrated in Fig. 5, our methodology consistently outperforms or matches the best alternative at every complexity level.

At simple complexity levels, our methodology achieves 100% success rate, demonstrating flawless performance on foundational DSL constructs. The enhanced context configurations (2-4) all perform exceptionally at this level (95-100%), while even the baseline achieves 93% success. Medium complexity introduces the first significant performance divergence, where our methodology maintains 92% success while the baseline drops dramatically to 68%, highlighting the critical value of enhanced context at moderate complexity levels.

High complexity demonstrates the robustness of our approach, with our methodology achieving 92% success compared to 79% for the baseline. This 13 percentage point advantage at high complexity validates the importance of enhanced context guidance for complex DSL generation tasks.

The most compelling assessment of our methodology is observed at ultra-high complexity level, where all approaches encounter significant performance degradation. However, our methodology exhibits remarkable resilience at these levels. While the baseline collapses to only 20 percent success, our methodology maintains a 75 percent success rate, representing a 55 percentage point advantage and nearly 4 times better performance than the minimal context approach.

These patterns directly answer **RQ2**, proving the methodology's consistency across complexity levels, with its advantages becoming more pronounced as DSL sophistication increases. The complexity-performance relationship reveals non-linear

degradation patterns characterized by performance plateaus followed by sharp degradation rather than gradual decline.

C. Critical Complexity Threshold Analysis

The ultra-high complexity results provide a definitive answer to **RQ3**, establishing that current LLM capabilities reach their practical limitations at approximately 30 requirements, but our methodology extends viable performance boundaries significantly beyond alternative approaches.

The magnitude of performance degradation at this threshold varies dramatically across context configurations. The baseline experiences a significant drop from 79% at high complexity to 20% at ultra-high complexity, representing a 59% point decline. In contrast, our methodology suffers a more manageable 17% drop from 92% to 75%, demonstrating superior stability at extreme complexity levels.

Our methodology's 75% success rate at ultra-high complexity represents a 25-55% advantage over alternative approaches at this (challenging) level. This substantial performance gap suggests that complete context guidance, particularly error prevention components, becomes increasingly valuable as complexity approaches LLM capability boundaries.

Obviously, conclusions are drawn based on the experiments performed on the specific types of DSLs. However, we argue that similar performance is expected in other types of DSLs (accuracy degradation when FR/NFR numbers increase) since the methodology is based on the complexity of the DSL (in terms of requirements) and not the domain itself.

D. Context Component Value Analysis

The progressive context enhancement design enables isolation of individual data component contributions to grammar generation performance. The experimental results reveal distinct value patterns for each context element:

Reference Grammar Impact: Adding example grammars (Configuration 1 - 2) provides the largest single improvement, particularly at medium complexity where it prevents a 25 percentage point performance drop observed in the baseline.

Model Example Value: The inclusion of concrete usage examples (Configuration 2 - 3) shows variable impact across complexity levels, with modest improvements at most levels but maintaining consistent performance.

Error Prevention Guidance: The addition of common syntax error guidance (Configuration 3 - 4) provides crucial performance gains, particularly at ultra-high complexity where it delivers a 10 percentage point improvement that proves essential for maintaining viable success rates.

E. Requirement Fulfilment Commentary

To evaluate the practical utility of syntactically valid grammars beyond technical correctness, manual analysis was conducted on a sample of successfully generated grammars, providing insights into the gap between syntactic correctness and complete requirement implementation.

The analysis reveals that most requirements are correctly implemented across all context configurations and complexity

levels, with LLMs demonstrating strong capability in translating natural language specifications into grammar constructs. Unlike the dramatic performance differences observed in syntax validation success rates, requirement fulfilment shows much more consistent patterns across configurations, with relatively smaller gaps between different approaches.

Systematic patterns emerge on how LLMs interpret ambiguous specifications, with consistent interpretive frameworks applied across different context configurations. When requirement omissions occur, they follow predictable patterns rather than random failures, with our methodology (Configuration 4) demonstrating particularly uniform behavior in requirement coverage. This uniformity means that when multiple grammars generated using our methodology fail to generate certain requirements, they consistently fail to generate the same specific requirements rather than exhibiting random omission patterns.

These findings highlight an important distinction between technical validity and specification compliance in automated grammar generation, while confirming that our methodology's syntax validation performance translates to reliable practical utility for DSL development workflows.

VII. DISCUSSION AND CONCLUSIONS

This work presents RQ2DSL, an AI agent implementing a systematic methodology for automated TextX grammar generation using Large Language Models. Experimental validation across 1,280 experiments demonstrates that RQ2DSL achieves superior performance compared to reduced-context alternatives, maintaining 92-100% success rates for moderate complexity DSLs and 93.4% overall success through integration of five context components. The agent's resilience at ultra-high complexity (75% vs 20% for minimal context) shows that systematic error prevention becomes increasingly critical as DSL sophistication approaches LLM capability boundaries. RQ2DSL excels in technical precision, directly providing usable DSL grammars for sets with 4-15 requirements. At ultra-high complexity, it provides the best available automated approach for hybrid human-AI development workflows. The methodology's modular approach enables adaptive implementation based on complexity requirements, while systematic requirement omission patterns create opportunities for automated quality assurance frameworks. The primary limitation is the sharp performance degradation at 30 requirements, representing a threshold where LLM capabilities reach their boundaries. While RQ2DSL maintains viable performance (75%) at this threshold, the drop from 92% indicates fundamental constraints in current language model architectures for handling highly complex grammar generation tasks.

VIII. FUTURE WORK

Evaluation of RQ2DSL across diverse LLM architectures and DSL-creation frameworks represents a critical next step for establishing broader applicability. Comparative studies involving different LLMs (GPT-4, Gemini, open-source alternatives) and grammar specification languages (ANTLR, Xtext, MPS) would validate the generalizability of RQ2DSL's

systematic approach and identify model-specific optimization strategies. The ultra-high complexity threshold identified at 30 requirements requires investigation across different architectures to understand whether this represents a fundamental limitation of current transformer models or a constraint that can be addressed through RQ2DSL enhancements. Domain-specific adaptations of RQ2DSL could reveal whether certain application areas benefit from specialized context components. Investigating vertical domains such as cyber-physical systems or robotics, may uncover domain-specific patterns that enhance RQ2DSL's effectiveness. Incorporating domain ontologies, specialized vocabulary and domain-specific grammar patterns into RQ2DSL's context framework could significantly improve generation quality for applications requiring temporal logic operators, control theory primitives, motion planning constructs, or communication protocols. Advanced RQ2DSL enhancements could explore adaptive context selection mechanisms that automatically adjust the context framework based on requirement complexity analysis, automated reference grammar selection from larger repositories and dynamic error prevention guidance that evolves based on failure pattern analysis. These extensions would further optimize RQ2DSL's performance while reducing the manual effort required for implementation across diverse DSL development scenarios.

IX. ACKNOWLEDGEMENTS

Parts of this work have been supported by the project GEAR-UP, co-funded by the European Union through the LIFE programme under Grant Agreement no. 101167702.

REFERENCES

- [1] M. Mernik, J. Heering, and A. M. Sloane, "When and how to develop domain-specific languages," *ACM Comput. Surv.*, vol. 37, no. 4, p. 316–344, Dec. 2005. [Online]. Available: <https://doi.org/10.1145/1118890.1118892>
- [2] M. Völter, S. Benz, C. J. Dietrich, B. Engelmann, M. Helander, L. C. L. Kats, E. Visser, and G. Wachsmuth, "Dsl engineering - designing, implementing and using domain-specific languages," 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:42569834>
- [3] C. Di Sipio, R. Rubel, J. Di Rocco, D. Di Ruscio, and L. Iovino, "On the use of llms to support the development of domain-specific modeling languages," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS Companion '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 596–601. [Online]. Available: <https://doi.org/10.1145/3652620.3687808>
- [4] F. Tomassetti and V. Zaytsev, "Reflections on the lack of adoption of domain specific languages," in *STAF 2020 Workshop Proceedings*, ser. CEUR Workshop Proceedings, L. Burgueño and L. Kristensen, Eds. Germany: CEUR, Oct. 2020, pp. 85–94, 5th International Workshop on Open and Original Problems in Software Language Engineering, OOPSLE 2020, OOPSLE 2020 ; Conference date: 22-06-2020 Through 26-06-2020. [Online]. Available: <http://oopsle.github.io/2020/index.html>
- [5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [6] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, P. S. Dulepet, S. Vidyadhara, D. Ki, S. Agrawal, C. Pham, G. Kroiz, F. Li, H. Tao, A. Srivastava, H. D. Costa, S. Gupta, M. L. Rogers, I. Goncearenco, G. Sarli, I. Galyner, D. Peskoff, M. Carpuat, J. White, S. Anadkat, A. Hoyle, and P. Resnik, "The prompt report: A systematic survey of prompt engineering techniques," 2025. [Online]. Available: <https://arxiv.org/abs/2406.06608>
- [7] B. Wang, Z. Wang, X. Wang, Y. Cao, R. A. Saurous, and Y. Kim, "Grammar prompting for domain-specific language generation with large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2305.19234>
- [8] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," 2023. [Online]. Available: <https://arxiv.org/abs/2305.06599>
- [9] I. Dejanović, R. Vadera, G. Milosavljević, and Ž. Vuković, "Textx: A python tool for domain-specific languages implementation," *Knowledge-Based Systems*, vol. 115, pp. 1–4, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950705116304178>
- [10] S. Ugare, T. Suresh, H. Kang, S. Misailovic, and G. Singh, "Syncode: Llm generation with grammar augmentation," 2024. [Online]. Available: <https://arxiv.org/abs/2403.01632>
- [11] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 9895–9901. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.779/>
- [12] F. Brad, R. Iacob, I. Hosu, S. Ruseti, and T. Rebedea, "A syntax-guided neural model for natural language interfaces to databases," in *2018 IEEE 30th International Conference on Tools with Artificial Intelligence (ICTAI)*, 2018, pp. 229–233.
- [13] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," 2023. [Online]. Available: <https://arxiv.org/abs/2303.17651>
- [15] A. Ojamaa, H.-M. Haav, and J. Penjam, "Semi-automated generation of dsl meta models from formal domain ontologies," in *Model and Data Engineering*, L. Bellatreche and Y. Manolopoulos, Eds. Cham: Springer International Publishing, 2015, pp. 3–15.
- [16] H. Krahn, B. Rumpe, and S. Völkel, "Monticore: a framework for compositional development of domain specific languages," 2014. [Online]. Available: <https://arxiv.org/abs/1409.2367>
- [17] T. Degueule, B. Combemale, A. Blouin, O. Barais, and J.-M. Jézéquel, "Melange: a meta-language for modular and reusable development of dsls," in *Proceedings of the 2015 ACM SIGPLAN International Conference on Software Language Engineering*, ser. SLE 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 25–36. [Online]. Available: <https://doi.org/10.1145/2814251.2814252>
- [18] J. Cabot, "Could an LLM create a full domain-specific language?" Online, Jun. 2025, accessed: Jan. 2025. [Online]. Available: <https://modeling-languages.com/vibe-dslng/>
- [19] K. Panayiotou, C. Doumanidis, E. Tsardoulas, and A. L. Symeonidis, "Smauto: A domain-specific-language for application development in smart environments," *Pervasive and Mobile Computing*, vol. 101, p. 101931, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1574119224000579>
- [20] Robotics-4-All, "goal-dsl," GitHub repository, accessed: 2025. [Online]. Available: <https://github.com/robotics-4-all/goal-dsl>
- [21] Anthropic, "Claude 3.7 sonnet and claude code," Anthropic News, 2025, accessed: 2025. [Online]. Available: <https://www.anthropic.com/news/claude-3-7-sonnet>